Ders 10 Genişletilmiş Notu: Gereksinim Belgesi Değerlendirmesi, Proje Grupları ve CMMI ile TDD Tekrarı

Ders 10 Genişletilmiş Notu: Gereksinim Belgesi Değerlendirmesi, Proje Grupları ve CMMI ile TDD Tekrarı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Projenin Bundan Sonraki Belgeleri
- 2. Doğru Bir Gereksinim Belgesi (SRS) Nasıl Olmalı?
- 3. Bir Grubun SRS Belgesi Üzerinden Değerlendirme
- 4. Gereksinim Analizinin Amacı: Problemi Anlamak
- 5. Gereksinim Mühendisliğinin Adımları
- 6. Use Case Senaryolarında Sorulacak Sorular
- 7. Projede Beklenen Test Türleri
- 8. CMMI Tekrarı
- 9. CMMI Düzeyleri ve Anahtar Süreç Alanları
- 10. People CMM Tekrarı
- 11. Çevik Yaklaşımlar, XP ve Eşli Programlama
- 12. Statik ve Dinamik Kalite Değerlendirmesi
- 13. Test Güdümlü Yazılım Geliştirme (TDD) Tekrarı
- 14. Sonraki Haftaların Konuları

1. Dersin Planı ve Projenin Bundan Sonraki Belgeleri

Hoca derse, bu haftanın iki ana amacı olduğunu söyleyerek başladı. Birincisi, grupların sınavdan önceki haftada gönderdiği gereksinim belgelerini (requirements document) değerlendirmekti. Hoca bunu her grubun hatasını tek tek söyleyerek değil, "doğru bir gereksinim raporu nasıl olmalı?" sorusu üzerinden yapmayı seçti. İkincisi, geçen hafta görülen yazılım süreç iyileştirme (software process improvement) ve test güdümlü yazılım geliştirme (Test Driven Development, TDD) konularını tekrar etmekti.

Hoca projenin bundan sonraki aşaması için ayrı bir rapor formatı vermeyeceğini belirtti. Proje planında olduğu gibi test planında hangi maddelerin bulunduğu derste görülmüştü. Gruplar bu maddelere göre bir test planı hazırlayacak, ardından gerçekleştirdikleri birim testi (unit test), sistem testi (system test) ve kullanıcı testini (user test) raporlayacaklar. Burada önemli bir bağlantı var: Sistem testi ve kullanıcı testi, grupların hazırladığı gereksinim spesifikasyonuna dayanarak yapılacak. Yani gereksinim belgesi yalnız bir ara teslim değil, testlerin de girdisidir.

2. Doğru Bir Gereksinim Belgesi (SRS) Nasıl Olmalı?

Hoca, daha önce de birlikte incelenen, başka bir öğrenci projesine ait bir yazılım gereksinim spesifikasyonu (Software Requirements Specification, SRS) örneğini yeniden açtı. Gruplardan çoğunun belgesi bu örneğe uygun hazırlanmıştı. Uygun hazırlamayanlar için örnek bir kez daha gözden geçirildi.

Örnek belgenin bölümleri şöyleydi:

- **İçindekiler tablosu.**
- **İçerik bilgisi:** Belgenin amacı ve özeti. Hoca küçük bir uygulama için bu bölümün çok gerekli olmadığını söyledi.
- **Ürünün tanımı:** Seçilen yazılım ne amaçla geliştirilmiş, hangi işleri gerçekleştiriyor? Bu genel tanım bir use case diyagramıyla gösterilebilir.
- **Fonksiyonel gereksinimler (functional requirements):** Sistemin aktörleri kimler, sistem sınırları neler, yazılım genel olarak hangi işleri yapıyor? Aktörler belirtilerek tanımlanır. Her use case için, senaryonun ayrıntılı belgelendiği bir tanım konabilir.
- **Fonksiyonel olmayan gereksinimler (non-functional requirements):** Kullanıcı arayüzü gereksinimleri, kullanılabilirlik (usability), güvenlik (security) gibi.

Hoca use case belgesi kullanmanın zorunlu olmadığını tekrarladı. Use case belgesindeki temel akışta (basic path) yer alan ana işlemlerin her biri tek bir madde olarak yazılabilir; fonksiyonel gereksinimler böylece maddeler hâlinde verilebilir. Yine de grupların çoğu use case kullanmış, sistemi modüllere ayırmış ve her modülde "yeni kayıt girişi", "kaydın güncellenmesi" gibi işlemleri ve bunların alternatif yollarını belirtmişti. Hoca, işlevsel gereksinimlerin bu şekilde adım adım yazılması gerektiğini vurguladı.

> **Ek bilgi (derste anlatılmadı):** SRS belgeleri için yaygın kullanılan şablon büyük olasılıkla IEEE 830 (IEEE Recommended Practice for Software Requirements Specifications) standardıdır. Bu şablon belgeyi giriş (amaç, kapsam, tanımlar), genel tanım (ürün perspektifi, kullanıcılar, kısıtlar) ve özel gereksinimler (fonksiyonel, arayüz, performans, diğer nitelikler) bölümlerine ayırır. Standart daha sonra ISO/IEC/IEEE 29148 ile güncellenmiştir.

3. Bir Grubun SRS Belgesi Üzerinden Değerlendirme

Hoca ardından rastgele seçtiği bir grubun SRS belgesini açtı. Belge, örnekteki gibi giriş, amaç ve kapsam gibi belgeyle ilgili tanımlamalarla başlıyor, genel sistem bilgisini veriyordu. Grup, sistemin genel bir use case diyagramını çizmiş, fonksiyonel gereksinimleri de use case diyagramlarıyla tanımlamıştı.

Hoca gruptan derste bulunan bir üyeye use case diyagramlarını neye göre bölüştürdüklerini sordu. Belgenin ağırlıkla başka üyeler tarafından hazırlandığı anlaşıldı. Hoca bu durumu sunumlar için bir uyarıya dönüştürdü: Gruptaki herkes yapılan işi anlamış ve anlatabilecek durumda olmalıdır. Sunumlarda her grup üyesine soru sorulabilir.

> **Hocanın vurgusu:** Grubun her üyesi aynı notu almayacak; projeye kim ne kadar sahip çıkmışsa notu ona göre olacak. Gönderdiğiniz belgeyi grubun herkesinin anlamış olup olmadığını kontrol edin, yaptığınızı "hazmetmeye" çalışın.

Hoca belgelerin çoğunun benzer şekilde hazırlandığını, örneği anlaşılırlık için gösterdiğini söyledi ve gereksinim konusuna geri döndü.

4. Gereksinim Analizinin Amacı: Problemi Anlamak

Hoca gereksinim belgelerini incelerken gereksinim analizi (requirements analysis) üzerinde biraz daha durması gerektiğini gördüğünü söyledi. Gereksinim analizini neden yapıyoruz? Öncelikle problemi anlayabilmek için.

Problemi anlamak için sorulacak sorular şunlardır:

- Bu problem neyi çözüyor?
- Bu çözümü kim kullanacak?
- Neler bilinmiyor? Bilinmeyenler var mı?
- Problemi çözmek için gerekli özellikler, veriler ve işlevler nelerdir?
- Problemi, daha iyi anlaşılacak daha küçük problemlere bölmek mümkün mü?
- Problemi grafik olarak gösterebilir miyiz? Bir analiz modeli kurabilir miyiz?

Bu aşamadan sonra yazılımın temel alt parçaları belirlenir. Bu alt parçalar daha sonra tasarımda modül olarak karşımıza çıkar.

Hoca projedeki durumun normal akışın tersi olduğuna dikkat çekti. Öğrencilerden bir yazılım seçmeleri istenmişti ve bu yazılım onlar tarafından geliştirilmemiş de olabilirdi. Yani elde hazır bir yazılım var ve onun gereksinim belgesi sonradan yazılıyor. Bu, sondan başa doğru gitmektir. Düz akışta önce verilen bir problemi anlamak için gereksinim analizi yapılır, sonra yazılım geliştirilir. Ama tersten gidilse bile ilk soru aynıdır: Bu yazılım hangi problemi çözüyor?

Hoca örnek olarak Software Engineering kitabındaki SafeHome ev güvenlik sistemini hatırlattı. Her yazılım projesinde olduğu gibi burada da önce "ev güvenlik sistemi hangi işi gerçekleştiriyor, nasıl bir ortamda çalışıyor?" sorularının cevabını anlamak gerekir.

5. Gereksinim Mühendisliğinin Adımları

Hoca gereksinim mühendisliği (requirements engineering) adımlarını Yazılım Mühendisliği dersinden hatırlattı. Amacı bu adımların ezber kalmamasıydı; her adımda ne yapıldığını anlamak gerekir.

5.1 Başlangıç (Inception)

Yeni başlanan bir uygulama için önce sorular sorulur:

- Temel problemi anladık mı?
- Bu çözümü isteyen insanlar ve kullanıcıları kimlerdir?
- Tasarlanması istenen çözümün yapısı ne olmalıdır?
- Müşteri ile geliştirici arasındaki iş birliği ve iletişim nasıl sağlanmalıdır? Karşımızda nasıl bir müşteri grubu var?

5.2 Gereksinim Toplama (Elicitation)

Bu adımda tüm paydaşlardan (stakeholders) gereksinimler toplanır. Paydaşlar kullanıcılar, sponsor, yöneticiler ve uç kullanıcılardır. Her birinin sistemden beklentileri ve istekleri sorulur. Yönetici bakış açısı, uç kullanıcı bakış açısı gibi farklı bakış açılarına göre, herkesin yaptığı iş kendisinden dinlenir.

Hocanın örneği bir satış sipariş uygulamasıydı. Bu uygulamayla yalnız satış bölümü ilgili değildir. Siparişleri alanlar, faturayı hazırlayanlar ve depodakiler de ilgilidir. Hem üretim hem satış bölümü stoklara da baktığı için hepsi bu uygulamanın paydaşıdır. Gereksinim toplamada hepsinden kendi yaptıkları işi anlatmaları istenir.

5.3 Ayrıntılandırma (Elaboration)

Toplanan bilgilerle sistemin modeli çıkarılır. Grup olarak bir model yapısı kararlaştırılır: use case diyagramı mı, veri akış diyagramı (data flow diagram) mı? Hedef, sisteme giren veriyi, işlemleri ve davranışları, yani bütün gereksinimleri modelleyebilmektir.

5.4 Pazarlık (Negotiation)

Hoca pazarlığın iki anlamı olduğunu anlattı:

- **Farklı tarafların hemfikirliği:** Bir uygulamada farklı taraflar görev alıyorsa, her biri kendi yaptığı işin modelde doğru gösterildiğini görmelidir. Sizin anladığınız ve çizdiğiniz model onlar açısından da uygun mu?
- **Kullanıcı ile geliştiricinin anlaşması:** Geliştirici kullanıcının isteklerini anlamalı ve "bunu yazılıma dökebilirim" konusunda kullanıcıyla anlaşmalıdır. Kullanıcı tarafının istekleri ile geliştiricinin yapabilecekleri arasındaki çakışma ve örtüşme bu adımda ele alınır.

5.5 Spesifikasyon (Specification)

Ancak bu aşamalardan geçtikten sonra spesifikasyon yazılabilir. Hoca, gruplardan istediği SRS belgesinin bütün bu süzgeçten geçtikten sonra hazırlanan belge olduğunu vurguladı. Bu adımda:

- Belge yazılır, modeller oluşturulur.
- Matematiksel bir formül varsa ifadesi geliştirilir.
- Sistem use case'lerle anlatıldıysa use case'ler gözden geçirilerek doğru biçimde yayımlanır.
- Gerekirse bir prototip çıkarılır; yani daha kolay yazılım elemanlarıyla çalışır hâlde bir model oluşturulur.

5.6 Geçerleme (Validation)

Geçerleme, SRS belgesi üzerinde yapılan bir gözden geçirme mekanizmasıdır. Bir üst adımda spesifikasyon yazılmıştı; şimdi o spesifikasyonda hata olup olmadığına bakılır. Sorulan sorular şunlardır:

- Sistemi doğru yorumladık mı?
- Belgenin bazı bölümlerini daha açık ifade etmek gerekiyor mu?
- Eksik bilgi (missing information) var mı? Olması gereken ama boşluk kalan yerler var mı?
- Büyük ürünlerde ya da sistemlerde alt modüller arasında uyumsuzluk var mı? Büyük sistemlerde çok sayıda alt modül vardır ve bunlar birleştirilirken uyumsuzluk çıkabilir.
- Gereksinimler arasında gerçekçi olmayan, başarılamayacak ya da birbiriyle çakışan, ters düşen anlatımlar var mı?

Hoca son maddeyi pazarlık adımına bağladı: Pazarlık doğru yapıldıysa gerçekçi olmayan gereksinim kalmamalıdır. Çünkü pazarlıkta kullanıcıların istekleri ile geliştiricilerin yapabilecekleri zaten örtüştürülmüştür.

> **Hocanın vurgusu:** Spesifikasyonu yazdıktan sonra mutlaka bir geçerleme çalışması yapmanız gerekir. Başlangıç, gereksinim toplama, ayrıntılandırma ve pazarlık adımları doğru yapılırsa spesifikasyon da doğru olur.

5.7 Gereksinim Yönetimi (Requirements Management)

Gereksinim yönetimi yeni bir adım değil, bütün bu adımların yönetimidir: Adımlar doğru planlandı mı, doğru yürütüldü mü, bütün çalışanlar doğru hareket etti mi, müşteriler ve kullanıcılarla doğru bir birlikte çalışma sağlandı mı?

> **Hocanın vurgusu:** Gereksinim analizinde hangi adımlar olduğu ve bu adımlarda ne yapıldığı, Yazılım Mühendisliği dersinde de çok sorulan bir bölümdür.
>
> **Ders sonrası not:** Altyazıda adımlar "inception, illustration, elaboration, negotiation" diye geçer. "Illustration" konuşma tanıma hatasıdır; doğrusu elicitation (gereksinim toplama).

6. Use Case Senaryolarında Sorulacak Sorular

Hoca öğrencilerin nesneye yönelik yaklaşım (object-oriented approach) konusunda önceki derslerden bilgi sahibi olduğunu hatırlattı. Use case'lerde senaryolar bir sistem kullanımını anlatır. Her senaryoda önce aktörün kim olduğu belirlenir. Ardından şu sorular sorulur:

- Birincil (esas) aktör kimdir? İkincil aktör kimdir?
- Aktörlerin hedefleri nelerdir?
- Bu hikâye başlamadan önce sağlanması gereken ön koşullar (preconditions) nelerdir?
- Her aktörün yapacağı işler nelerdir?
- Tanımlanan işleri genişletmek mümkün mü? Bazı durumlarda alternatif yollar var mı?
- Her aktör sistemde neyi isteyecek, üretecek ya da değiştirecek? Sistemde hangi değişiklikler olabilir?
- Beklenmeyen durumlarda ve beklenmeyen değişikliklerde ne yapılmalıdır?

Hoca özellikle alternatif senaryoların da use case'lerde düşünülmesi gerektiğini vurguladı. Yalnız her şeyin yolunda gittiği temel akışı yazmak yetmez.

7. Projede Beklenen Test Türleri

Proje gruplarının kontrolü sırasında hoca, gruplara beklenen testleri açıkladı. Bir öğrencinin sorusu üzerine bunu bütün sınıf için tekrarladı.

Test	Kapsamı
Birim testi (unit test)	En az bir birim testi
Sistem testi (system test)	Fonksiyonel olmayan bir gereksinim için; açık kaynaklı bir araçla; güvenlik ya da performans gibi bir konu seçilebilir
Kullanıcı testi (user test)	Fonksiyonel gereksinimlerin kullanıcı bakış açısıyla nasıl sağlandığı; gereksinim analizine (SRS) dayanır

Hoca sistem testiyle ilgili bir noktayı açtı. Gruplar yazılımın bütününü test edemeyebilir; bu yüzden bütünlük testi (integration test) istenmiyor. Ama belli bir modül içinde de olsa bir sistem testi yapılmalıdır. Çünkü fonksiyonel olmayan özellikler gereksinim analizinde tanımlandı ve bunların sağlanıp sağlanmadığı ancak sistem testiyle görülebilir. Bu testi yapabilmek için çoğu zaman bir araç gerekir. Hoca grupların görev paylaşımı yaparak açık kaynaklı bir araç araştırmasını önerdi.

Kullanıcı testinde ise gereksinimlerdeki fonksiyonel özelliklerin, kullanıcının bakış açısıyla bakıldığında nasıl sağlandığı gösterilmelidir. Bu yüzden kullanıcı testi doğrudan gereksinim belgesine göre yapılır.

8. CMMI Tekrarı

Aradan sonra hoca, geçen hafta görülen süreç iyileştirme konusunu tekrar etti. CMMI (Capability Maturity Model Integration), kalite iyileştirme konusunda yazılım geliştirme sürecini iyileştirmek için yapılması gerekenleri tanımlar. Hoca CMMI süreç modelinin yeterlik düzeylerinin ve yazılım geliştirme süreciyle anahtar süreç alanları arasındaki ilişkinin sınav kapsamı dışında kalan, vize sonrasına bırakılan kısım olduğunu, bu yüzden geçen hafta üzerinde durulduğunu söyledi.

Yazılım geliştirme ortamı için sertifika sistemleri olarak CMMI ve SPICE görülmüştü. Hoca ISO'da da yazılım geliştirme süreciyle ilgili tanımlamalar bulunduğunu hatırlattı. CMMI, ürünün kalitesini nasıl sağlayacağımızı tanımlayan bir sistemdir ve etkin bir yazılım süreci için anahtar elemanlara sahiptir.

8.1 Süreç İyileştirme Döngüsü

Süreç iyileştirme bir döngüdür:

- Önce ölçüm yapılır.
- Ölçümler üzerinde analiz yapılır.
- Sistemde değişiklik olmuşsa yeni ölçümler yapılır.
- Bu ölçümlerin karşılığı yeniden analiz edilir ve döngü devam eder.

Bunun anlamı şudur: CMMI 2 sertifikası alan bir kurum hep orada kalmak zorunda değildir. Belirli bir süre sonra gerekli koşulları yerine getirip 3. düzeye çıkabilir ya da 2. düzeyi koruyamayabilir. Değişime göre ölçümler sürekli devam eder.

Hoca bunu üniversite bölümlerinin kalite sertifikalarına benzetti. MÜDEK akreditasyonu almış bir bölüm beş yıl sonra yeniden değerlendirilir; hazırlık yaparak tekrar alabilir ya da alamayabilir. ABET de benzer şekilde uluslararası bir akreditasyondur.

8.2 CMM ile Farkı

CMMI'de proje yönetimi, mühendislik yönetimi ve tüm süreç yönetimi bütünleşik olarak sertifikasyondan geçer. Bir kurum birinde 2., birinde 3. düzeyde olamaz; hepsinde belli bir düzeyi sağlaması gerekir.

Ders sonrası not: Altyazıda “CMMI, CMMI’den farklı olarak” geçer. Kastedilen karşılaştırma CMM ile CMMI arasındadır: CMMI, CMM’den farklı olarak alanları bütünleşik değerlendirir.

9. CMMI Düzeyleri ve Anahtar Süreç Alanları

9.1 Yeterlik Düzeyleri

CMMI’nin beş düzeyi şunlardır:

- **1. Başlangıç (Initial):** Gelişigüzel ve kaotik bir yapı vardır. Plan yapılsa da terk edilebilir. Yazılım sürecindeki yetenekler önceden kestirilemez.
- **2. Yönetilen (Managed):** Sertifikalandırılan ilk düzeydir. Proje yönetimi kontrolleri oluşturulur, proje gereksinimleri yönetilir, ürün ve hizmetler yönetilir. Gereksinimler belli standartları sağlamalıdır.
- **3. Tanımlı (Defined):** Süreçler iyi tanımlanmıştır; standartlar, prosedürler, araçlar ve metotlarla anlatılmıştır. Standart süreç tanımlarıyla kurumsal boyutta tutarlılık sağlanır. Tüm projelerde standart yazılım geliştirme sürecinin uyarlanmış hâli kullanılır. Yani mutlaka bir yazılım geliştirme süreç modeli kullanılır ve standartlara göre ilerlenir.
- **4. Sayısal Yönetilen (Quantitatively Managed):** Sürecin başarımı için önemli alt süreçler seçilir ve istatistiksel ve diğer sayısal tekniklerle kontrol edilir. Kalite ve süreç performansı için sayısal hedefler konur ve süreç yönetiminde ölçüt olarak kullanılır. Süreçler ölçülür; ölçülebilen sınırlar içinde işler gerçek verilere dayanarak tahmin edilir.
- **5. En İyileştiren (Optimizing):** Süreçten gelen sayısal geri beslemeler ve teknolojilerden yararlanılarak sürekli süreç iyileştirmesi yapılır. Yenilik yapmak da önemlidir.

Hoca 4. düzeyle ilgili sık yapılan bir yanılgıyı düzeltti. “Sayısal yönetilen” demek yalnız ölçüm yapılması demek değildir. Yazılım ölçümü 2. düzeyden itibaren yapılır. 4. düzeyin farkı, yönetimin bu ölçümlere dayanarak yapılmasıdır. 5. düzeyi 4. düzeyden ayıran ise süreçlerin değişme derecesi ve sürekli yeniliktir.

9.2 Anahtar Süreç Alanları

Hoca CMMI süreç alanlarını anahtar süreç alanlarıyla (key process areas) eşleştirmek gerektiğini hatırlattı. Bunun için Yazılım Mühendisliği dersindeki aktivite ayrımına döndü:

- **Temel aktiviteler:** analiz, tasarım, gerçekleştirme, test.
- **Destek aktiviteleri:** konfigürasyon yönetimi, risk yönetimi, proje yönetimi, proje izleme ve kontrol.

CMMI’nin hangi düzeyinde hangi aktivitelerin gerçekleştirildiği, düzeyleri tanımlar:

- **2. düzey:** gereksinim yönetimi, proje planlama, proje izleme ve kontrol, tedarikçi sözleşme yönetimi, ölçüm ve mutlaka konfigürasyon yönetimi. Bunlar yapılır, ama her proje için ayrı ayrı yapılır.
- **3. düzey:** Kurumsal standartlaşma bu düzeyde başlar. Gereksinim geliştirme, ürün entegrasyonu (bütünleşmesi), doğrulama ve onaylama (verification and validation), kurumsal süreç tanımlama ve kurumsal süreç odaklanma, kurumsal eğitim, bütünleşik proje yönetimi (birçok projeyi birleştirerek yönetme), risk yönetimi.
- **4. düzey:** kurumsal süreç başarımı ve sayısal proje yönetimi. Sayısal proje yönetimi, ölçülenleri bir arşiv hâline getirip bu arşivden yararlanarak yeni projelerin iş gücü, maliyet ve süresi hakkında tahmin yapabilmektir.
- **5. düzey:** kurumsal yenilenme ve yaygınlaştırma, nedensel analiz (neden-sonuç analizi, causal analysis) ve yeniliklerin koordinasyonu.

CMMI’deki süreç alanları süreç yönetimi, proje yönetimi, mühendislik ve destek olarak dört grupta düşünülmelidir. Destek süreç alanları da ikiye ayrılır:

- **Temel destek süreç alanları:** ölçümleme ve analiz, kalite sağlama, konfigürasyon yönetimi.
- **İleri destek süreç alanları:** nedensel analiz ve çözümleme, karar analizi, kurumsal bütünleştirme altyapısı.

10. People CMM Tekrarı

People CMM, Software Engineering Institute (SEI) tarafından geliştirilmiştir. CMMI'nin alanları süreç, proje yönetimi, mühendislik ve destek adımlarıydı. İnsan da bunun içine girer; hocaya göre People CMM şu anda CMMI'nin içindedir.

İnsan tarafında olgunluk ve yetenek düzeyi belirlenirken şunlara bakılır:

- Yazılım geliştirme yeteneğini anlamak.
- İş gücü yeteneklerinin nasıl artırılacağını belirlemek.
- Kurumun hedefi ile çalışanların hedefini bir arada tutmak.
- İnsan değerlerini gözden geçirmek.

People CMM düzeyleri hocanın anlatımına göre şöyledir:

- **Tekrarlanabilir düzey (2. düzey):** tazminat, eğitim, performans yönetimi, personel alımı, çalışma ortamının düzenlenmesi. Firma bunlara odaklanmışsa ve bunu ispat edebiliyorsa 2. düzeydedir.
- **Tanımlı düzey (3. düzey):** katılımcı kültür, kariyer geliştirme, yetenek geliştirme, iş gücü planlama.
- **Yönetilen düzey (4. düzey):** kurumsal performans ayarlama, ekip temelli pratikler, takım kurma, danışmanlık.
- **En iyileştiren düzey (5. düzey):** inovasyon ve yaratıcılık; personel açısından koçluk ve kişisel yetenek geliştirme. Hoca bunun CMMI'deki 5. düzeyin yenilikçilik vurgusuyla aynı olduğunu belirtti.

Bu sertifikasyon yalnız yazılım evleri için değildir; bir şirketin içindeki yazılım geliştirme ekibi de alabilir. Hoca örnek olarak Siemens'in yazılım birimini verdi. Bu birimin sertifika aldığını, son yıllarda 5. düzeye çıkmış olabileceğini emin olmadan söyledi. Türkiye'de de birçok firmanın bu sertifikasyon adımlarından geçtiğini hatırlattı.

> **Ek bilgi (derste anlatılmadı):** People CMM'nin ilk sürümünde (büyük olasılıkla 1995) düzey adları Initial, Repeatable, Defined, Managed ve Optimizing idi; hocanın kullandığı "tekrarlanabilir" ve "yönetilen" adları bu sürümden gelir. İkinci sürümde (büyük olasılıkla 2001) düzeyler Initial, Managed, Defined, Predictable ve Optimizing olarak yeniden adlandırılmıştır. Düzeylerin içerdiği uygulamalar, yani tazminat, kariyer geliştirme, mentorluk ve sürekli yetenek iyileştirme gibi alanlar, büyük ölçüde aynı kalmıştır.

11. Çevik Yaklaşımlar, XP ve Eşli Programlama

11.1 Çevik Yaklaşımların Ortaya Çıkışı

Çevik yaklaşımlar (agile approaches), artan müşteri taleplerini daha hızlı karşılamak için geleneksel yazılım geliştirme süreç modellerine alternatif olarak ortaya çıktı. Daha önce görüldüğü gibi XP (Extreme Programming), Scrum ve Kanban gibi modeller vardır.

Hoca bu modellerin basitlik, iletişim, geri besleme ve cesaret olmak üzere dört temel değer çerçevesinde çalıştığını söyledi. Ortak özellikler şunlardır:

- küçük ve kısa aralıklı sürümler (small releases),
- basit tasarım ve devamlı yeniden tasarım (refactoring),
- ortak kod mülkiyeti (collective code ownership),
- sürekli tümleştirme (continuous integration),
- haftada 40 saat çalışma, yani devam ettirilebilir bir hız,
- müşterinin ekip üyesi olması (Scrum, XP ve Kanban'da),
- kodlama standartları.

> **Ders sonrası not:** Hoca bir yerde çevik dönemde geliştirilen süreç modellerinin genel adının XP olduğunu söyledi. Genel ad çevik (agile) yöntemlerdir; XP bunlardan biridir. Ayrıca hoca basitlik, iletişim, geri besleme ve cesaret değerlerinin bütün çevik modellerde bulunduğunu söyledi. Bu dört değer XP'nin değerleridir; XP'nin ikinci sürümünde beşinci değer olarak saygı (respect) eklenmiştir. Çevik Manifesto'nun dört değeri ise farklıdır.

11.2 Eşli Programlama

Hoca derste gelen bir soru üzerine eşli programlamayı (pair programming) açtı. Eşli programlama XP'nin temel özelliklerinden biridir. Çevik yöntemler 2000'li yılların başında tanıtıldığında ilk kullanılan XP idi ve eşli programlama en çok burada kullanıldı. Diğer çevik yöntemlerde "böyle çalışın" diye bir zorunluluk yoktur.

Eşli programlamada ayrı bir test grubu olmasın diye ya da test işi geliştirme sırasında yapılsın diye, kod yazan kişinin yanında bir kişi daha bulunur. Hoca bunun için tek ekran ve iki klavyeden oluşan XP çalışma ortamlarının bile çıktığını anlattı.

> **Hocanın vurgusu:** Eşli programlama XP'ye özeldir.

12. Statik ve Dinamik Kalite Değerlendirmesi

Geleneksel süreç modellerine göre çevik modellerde dinamik kalite değerlendirmesi, statik değerlendirmeden daha çok kullanılır. Yani test daha fazladır.

Hoca burada sınıfa bir soru sordu: Hangi yazılım kalite iyileştirme aktiviteleri statik değerlendirme yapar? Bir öğrenci kod gözden geçirme (code review) ve incelemeyi (inspection) örnek verdi. Hoca cevabı onayladı: İnceleme statik bir değerlendirmedir.

Çevik yaklaşımlarda kaliteyle ilgili bütün aktiviteler daha kısa aralıklarla, yani daha sık yapılır. Kullanıcı hikâyeleri (user stories), birim testler, müşteri geri beslemesi ve süreklilik, çevik yaklaşımları geleneksel modellerden ayıran özelliklerdir.

13. Test Güdümlü Yazılım Geliştirme (TDD) Tekrarı

13.1 Tanım ve Normal Akıştan Farkı

Test güdümlü (test yönelimli) yazılım geliştirme, çevik metotlarla birlikte kullanılır. Bu kalite stratejisinde küçük ve hızlı iterasyonlarla, yazılım geliştirme araçlarıyla hazırlanmış testler yazılır. TDD genellikle çevik yaklaşım uygulamalarına uygundur.

Hoca farkı normal akışla karşılaştırarak anlattı. Normal akışta yazılımın başında gereksinim analizi yapılır; use case diyagramları ya da başka modeller oluşturulur ve gereksinimler belirlenir: Bu sistem nasıl çalışır, neye ihtiyacı vardır? TDD'de ise gereksinim belirlenir belirlenmez "bu gereksinimi nasıl bir senaryoyla test ederim?" sorusu üzerinde de çalışılır. Test, geliştirmenin sonuna bırakılmaz; geliştirmeyle birlikte yürür.

13.2 TDD Akışı

Yazılım mühendisleri TDD ile kodu şöyle geliştirir:

- Bir use case diyagramı çizilir ve use case senaryosu, yani gereksinim senaryosu oluşturulur.
- Bu gereksinimi sağlamak için nasıl bir test senaryosu gerçekleştirileceği planlanır.
- Önce birim test case'leri yazılır. Henüz kod yazılmadığı için bu testler çalıştırılıp sonucu görülür.
- Bu planı sağlayacak kod yazılır ve test case'lerin geçmesi sağlanır.
- Test case'ler çalıştırıldığında hata varsa kod yeniden düzenlenir.

> **Ek bilgi (derste anlatılmadı):** TDD döngüsü genellikle "kırmızı, yeşil, yeniden düzenle" (red, green, refactor) diye özetlenir. Kırmızı: Yeni test yazılır ve kod henüz olmadığı için başarısız olur. Yeşil: Testi geçirecek en basit kod yazılır. Yeniden düzenle: Testler geçer durumda kalarak kodun yapısı iyileştirilir.

13.3 TDD Üzerine Bir Çalışma

Hoca TDD üzerine yapılan ilk çalışmalardan birinin Hakan Erdoğmuş'a ait olduğunu söyledi. Bu araştırmaya göre TDD programcı üretkenliğini artırdı. Kalitede çok fark görülmedi, ama hız kazandırdı. Hocaya göre bu yüzden hem çevik yöntemler hem TDD yaklaşımı şirketlerde çok kullanım alanı buldu.

Ek bilgi (derste anlatılmadı): Hocanın andığı çalışma büyük olasılıkla Erdoğmuş, Morisio ve Torchiano'nun 2005'te IEEE Transactions on Software Engineering'de yayımlanan, öğrencilerle yapılmış test-önce (test-first) deneyidir. Bu çalışmada test-önce yaklaşımını kullananların ortalamada daha çok test yazdığı ve daha çok test yazanların daha üretken olma eğiliminde olduğu bildirilmiş, dış kalitede anlamlı bir fark bulunmamıştır.

Ders sonrası not: Altyazıda "test driving development" ve "test drive on development" geçer. Doğrusu Test Driven Development'tır.

14. Sonraki Haftaların Konuları

Hoca sonraki haftalarda işlenecek konuları sıraladı:

- **Konfigürasyon yönetimi ve sürüm yönetimi (release management):** Konfigürasyon yönetimi daha önce yalnız tanım olarak görülmüş ve bir kalite sağlama aktivitesi olduğu belirtilmişti. Önümüzdeki hafta sürüm yönetimiyle birlikte ayrıntılı işlenecek.
- **Bakım yönetimi:** Daha önce değinilen bu konu üzerinde yeniden durulacak.
- **Yeniden mühendislik (software re-engineering) ve tersine mühendislik (reverse engineering):** Yazılım Mühendisliği dersinin son haftalarında bazen görülen bu iki kavramdan hangisinin hangisinin içinde olduğu tartışılacak.

Örnek Sorular

Soru 1 (derste sözlü, 00:50:55): Gereksinim spesifikasyonunda use case kullanmak gerekli mi?

Çözüm (derste verildi): Hayır. Fonksiyonel gereksinimler net maddeler hâlinde de yazılabilir. Use case belgesindeki temel akışın her ana işlemi tek bir madde olabilir. Büyük sistemlerde gereksinimleri 1'den 50-70'e kadar tek liste yerine modül modül ayırmak daha iyidir.

Soru 2 (derste sözlü, 01:05:05): Projede hangi testler bekleniyor; bütünlük testi şart mı?

Çözüm (derste verildi): En az bir birim testi, açık kaynaklı bir araçla fonksiyonel olmayan bir gereksinim için bir sistem testi ve fonksiyonel gereksinimlerin kullanıcı bakış açısıyla sağlandığını gösteren, SRS'e dayanan bir kullanıcı testi. Bütünlük testi şart değildir; sistem testi tek bir modül içinde de yapılabilir.

Soru 3 (derste sözlü, 01:50:34): Eşli programlama bütün çevik yöntemlerde kullanılır mı?

Çözüm (derste verildi): Hayır. Eşli programlama XP'ye özeldir. Çevik yöntemler 2000'li yılların başında tanıtıldığında ilk kullanılan XP idi ve eşli programlama en çok orada kullanıldı. Diğer çevik yöntemlerde böyle bir zorunluluk yoktur.

Soru 4 (derste sözlü, 01:52:34): Hangi yazılım kalite iyileştirme aktiviteleri statik değerlendirme yapar? Örnek verin.

Çözüm (derste verildi): Kod gözden geçirme (code review) ve inceleme (inspection). Çevik modellerde ise ağırlık dinamik değerlendirmededir, yani testtedir.

Dersin Toparlaması

Dersin ilk yarısı gereksinim belgesinin nasıl yazılması gerektiğine ayrıldı. Bir SRS belgesi ürünün tanımını, aktörleri ve sistem sınırlarıyla fonksiyonel gereksinimleri ve kullanılabilirlik, güvenlik gibi fonksiyonel olmayan gereksinimleri içerir. Fonksiyonel gereksinimler use case'lerle ya da net maddelerle yazılabilir. Ama bu belge ancak başlangıç, gereksinim toplama, ayrıntılandırma ve pazarlık adımlarından geçtikten sonra yazılabilir ve yazıldıktan sonra mutlaka geçerlenmelidir. Use case'lerde aktörler, hedefler, ön koşullar, alternatif yollar ve beklenmeyen durumlar düşünülmelidir. Bu belge projede sistem testi ve kullanıcı testi için de temel olacaktır.

İkinci yarıda CMMI'nin beş düzeyi ve her düzeydeki anahtar süreç alanları, People CMM düzeyleri, çevik yaklaşımların ortak özellikleri, XP'ye özgü eşli programlama, statik ve dinamik

kalite değerlendirmesi ve TDD tekrar edildi. Önümüzdeki hafta destek aktivitelerinden konfigürasyon yönetimi ve sürüm yönetimi ile bakım, yeniden mühendislik ve tersine mühendislik konularına geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.